

Flutter & Dart de zéro à l'app

Un cours condensé pour apprendre le langage Dart et le framework Flutter, et construire vos premières interfaces mobiles multiplateformes.

 **Flutter**
SDK

 **Dart**
3

1 Qu'est-ce que Flutter & Dart ?

Comprendre l'écosystème avant d'écrire la moindre ligne de code.

Dart, le langage

Dart est un langage orienté objet, développé par Google, avec typage fort et syntaxe proche du Java/Kotlin/TypeScript. Il compile en code natif (ARM/x64) pour de bonnes performances, et supporte le *hot reload* pendant le développement.

Flutter, le framework

Flutter est un framework UI open-source qui permet d'écrire une seule base de code pour Android, iOS, Web, Windows, macOS et Linux. Contrairement à React Native, Flutter ne passe pas par des composants natifs : il dessine chaque pixel lui-même via son propre moteur graphique (Skia / Impeller), garantissant un rendu identique sur toutes les plateformes.

Un seul code source

Hot Reload

Tout est Widget

Rendu 60-120 fps

Installer son environnement

- 1 Télécharger le **Flutter SDK** depuis flutter.dev et l'ajouter au PATH.
- 2 Installer un éditeur : **VS Code** ou **Android Studio** avec le plugin Flutter/Dart.
- 3 Lancer `flutter doctor` pour vérifier que tout est bien configuré.
- 4 Créer un projet avec `flutter create mon_app`, puis `cd mon_app && flutter run`.

ASTUCE

Utilisez `flutter doctor -v` pour un diagnostic détaillé si un outil manque (Android SDK, Xcode, etc.).

STRUCTURE D'UN PROJET

`lib/main.dart` contient le point d'entrée. Le dossier `lib/` regroupe tout votre code Dart ; `pubspec.yaml` gère les dépendances et assets.

2 Syntaxe & bases de Dart

Variables, types, fonctions et structures de contrôle.

Variables & types

```
// Typage explicite
int age = 25;
double prix = 9.99;
String nom = 'Alice';
bool actif = true;

// Inférence de type
var ville = 'Paris';

// Immuable
final id = 42;
const pi = 3.14;
```

Null safety

```
// Nullable avec ?
String? surnom;

// Opérateur ??
print(surnom ?? 'Anonyme');

// Assertion non-null
surnom!.length;

// Attribution conditionnelle
surnom ??= 'Invité';
```

Fonctions

```
int addition(int a, int b) {
  return a + b;
}

// Fonction fléchée
int carre(int x) => x * x;

// Paramètres nommés (optionnels)
saluer({String nom = 'ami'}) => print('Bonjour $nom');
saluer(nom: 'Karim');
```

Collections & contrôle de flux

```
List<String> fruits = ['pomme', 'kiwi'];
Map<String, int> scores = {
  'Ana': 12, 'Sam': 9
};
Set<int> uniques = {1, 2, 3};
```

```
for (var f in fruits) {
  print(f);
}
if (scores['Ana']! > 10) {
  print('Bien joué');
} else {
  print('Continue');
}
```

3 Classes, héritage & asynchrone

Dart est un langage 100% orienté objet : même les fonctions sont des objets.

Déclarer une classe

```
class Animal {  
  String nom;  
  int age;  
  
  Animal(this.nom, this.age); // constructeur  
  
  void crier() => print('$nom fait du bruit');  
}  
  
class Chien extends Animal {  
  Chien(String nom, int age) : super(nom, age);  
  
  @override  
  void crier() => print('$nom aboie !');  
}
```

CONCEPTS CLÉS

- **Constructeurs nommés** : `Animal.bebe(this.nom) : age = 0;`
- **Classes abstraites** et **interfaces implicites** via `implements`
- **Mixins** avec `with` pour réutiliser du code entre classes

Asynchrone : Future & async/await

```
Future<String> recupererDonnees() async {  
  await Future.delayed(Duration(seconds: 2));  
  return 'Données reçues';  
}  
  
void main() async {  
  print('Chargement...');  
  var resultat = await recupererDonnees();  
  print(resultat);  
}
```

POURQUOI C'EST ESSENTIEL

Les appels réseau, la lecture de fichiers ou l'accès à une base de données sont asynchrones. Flutter s'appuie énormément sur `Future` et `Stream` pour ne jamais bloquer l'interface utilisateur.

4 Tout est Widget

En Flutter, chaque élément visible (et même invisible, comme le padding) est un widget.

Votre première application

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('Ma Première App')),
        body: Center(child: Text('Bonjour Flutter !')),
      ),
    );
  }
}
```

StatelessWidget vs StatefulWidget

StatelessWidget	StatefulWidget
Immuable, ne change pas après construction	Possède un état interne mutable
Idéal pour texte statique, icônes, mises en page fixes	Idéal pour compteurs, formulaires, animations
Une seule classe	Une classe Widget + une classe State

```
class Compteur extends StatefulWidget {
  @override
  State<Compteur> createState() => _CompteurState();
}

class _CompteurState extends State<Compteur> {
  int valeur = 0;
  void incrementer() => setState(() => valeur++);

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: incrementer,
      child: Text('Valeur : $valeur'),
    );
  }
}
```

5 Construire des interfaces

Les widgets de layout organisent l'espace : disposition, alignement, contraintes.

Arbre de widgets typique

Scaffold

```
└─ AppBar
└─ body: Column
    └─ Container (bannière)
    └─ Row
        └─ Icon
        └─ Text
    └─ Expanded → ListView
        └─ ListTile (x N)
```

Row & Column

```
Column(
  mainAxisAlignment:
    MainAxisAlignment.center,
  children: [
    Text('Titre'),
    Row(
      children: [
        Icon(Icons.star),
        Text('4.8'),
      ],
    ),
  ],
)
```

Container & padding

```
Container(
  padding:
    EdgeInsets.all(16),
  margin:
    EdgeInsets.symmetric(
      horizontal: 8),
  decoration: BoxDecoration(
    color: Colors.blue[50],
    borderRadius:
      BorderRadius.circular(12),
  ),
  child: Text('Carte'),
)
```

Widgets essentiels à connaître

Widget	Rôle
Expanded / Flexible	Répartir l'espace disponible dans un Row/Column
Stack	Superposer des widgets (positionnement libre)
ListView.builder	Listes longues et performantes (lazy loading)
GridView	Grilles d'éléments (galeries, catalogues)
SizedBox	Espacement fixe ou contrainte de taille

PIÈGE FRÉQUENT

Une Column à l'intérieur d'un Row non contraint provoque une erreur "RenderFlex overflowed". Utilisez Expanded ou Flexible pour borner la taille.

6 Gérer l'état de l'application

Du simple `setState` aux solutions robustes pour les grandes applications.

Niveau 1 : `setState` (état local)

Parfait pour un état limité à un seul widget (ex : un formulaire, un interrupteur). Simple mais ne se partage pas facilement entre écrans éloignés.

Niveau 2 : remontée d'état (lifting state up)

On déplace l'état vers le parent commun et on le transmet via des *callbacks* et des paramètres — la base de tout, avant d'ajouter une librairie.

Niveau 3 : solutions dédiées

Solution	Quand l'utiliser
Provider	Simple, recommandé pour débiter avec un état partagé
Riverpod	Successeur de Provider, testable, sans BuildContext
Bloc / Cubit	Architecture stricte, séparation logique/UI, gros projets
GetX	Tout-en-un (état, routes, DI), rapide à mettre en place

Exemple avec Provider

```
class Panier extends ChangeNotifier {  
  int total = 0;  
  void ajouter(int prix) {  
    total += prix;  
    notifyListeners(); // avertit les widgets abonnés  
  }  
}  
  
// Dans le widget :  
final panier = context.watch<Panier>();  
Text('Total : ${panier.total}');
```

RECOMMANDATION

Débutez avec `setState`, passez à **Provider** dès que plusieurs écrans doivent partager une donnée, et envisagez **Riverpod** ou **Bloc** pour les applications à grande échelle.

7 Navigation & appels réseau

Passer d'un écran à l'autre et consommer une API REST.

Navigator : changer d'écran

```
// Aller vers un écran
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) =>
      DetailPage(),
  ),
);
```

```
// Revenir en arrière
Navigator.pop(context);

// Routes nommées
Navigator.pushNamed(
  context, '/profil');
```

Appeler une API avec le package http

```
import 'package:http/http.dart' as http;
import 'dart:convert';

Future<List<String>> chargerUtilisateurs() async {
  final reponse = await http.get(
    Uri.parse('https://api.exemple.com/users'));

  if (reponse.statusCode == 200) {
    final data = jsonDecode(reponse.body);
    return List<String>.from(data.map((u) => u['nom']));
  } else {
    throw Exception('Échec du chargement');
  }
}
```

Afficher un résultat asynchrone : FutureBuilder

```
FutureBuilder<List<String>>(
  future: chargerUtilisateurs(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return CircularProgressIndicator();
    } else if (snapshot.hasError) {
      return Text('Erreur : ${snapshot.error}');
    }
    return ListView(children: snapshot.data!.map((n) => Text(n)).toList());
  },
)
```


8 Bonnes pratiques & ressources

Les réflexes à adopter pour écrire du code Flutter propre et maintenable.

Organisation du projet

- Séparer screens/, widgets/, models/, services/
- Découper l'UI en petits widgets réutilisables
- Extraire les couleurs/textes dans un thème central (ThemeData)

Tests

- **Unit tests** pour la logique métier (Dart pur)
- **Widget tests** pour vérifier le rendu d'un widget isolé
- **Integration tests** pour parcourir l'app comme un utilisateur

Performance

- Utiliser `const` devant les widgets qui ne changent pas
- Préférer `ListView.builder` aux listes construites entièrement
- Éviter les `setState` trop englobants

Publication

- `flutter build apk` / `appbundle` pour Android
- `flutter build ipa` pour iOS (nécessite Xcode)
- `flutter build web` pour un déploiement web

Prochaines étapes

Vous avez maintenant les bases pour construire une vraie application. Continuez avec :

- La documentation officielle : **docs.flutter.dev**
- Le catalogue de widgets interactif : **flutter.dev/docs/development/ui/widgets**
- Pub.dev pour explorer les packages (state management, animations, HTTP...)
- Construire un petit projet complet : liste de tâches, app météo, ou lecteur d'actualités